ikerlan

MEMBER OF BASQUE RESEARCH & TECHNOLOGY ALLIANCE



User Guide – SDK Fw.

Status of the procedure:

Author	Reviewed by	Approved by
Hebert Luyo	Ane Mijangos	Ane Mijangos
16/03/2026	16/03/2026	16/03/2026

INDEX

1. Ikerlan 3
2. SDK Overview 4
2.1. Software Architecture overview 4
2.2. Supported products 5
3. SDK Layers 5
3.1. Application Layer 5
3.2. Middleware Layer 6
3.3. Services Layer 8
3.4. Portability Layer 8
3.5. Client Platform Libraries 8
4. SDK Package description 9
5. SDK Public API 9
5.1. API Usage Model 9
5.2. Status Codes 10
5.3. Initialize Function 10
5.4. Device Mode Functions 11
5.4.1. SGR_SDK_Authenticate_Via_GW 11
5.5. Gateway Mode Functions 11
5.5.1. Token-to-Server Authentication 11
5.5.2. Server-to-Token Authentication 12
5.5.3. SGR_SDK_GW_Handle_Device_Auth 13
6. Configuration 14
6.1. SDK Mode 14
6.2. Enabling Debug Output 14
6.3. Retry Configuration 15
6.4. CRC Validation 15
7. Proof of Concept (PoC) 16
7.1. PoC Scope and Purpose 16
7.2. Architecture 16
7.3. PoC Dependencies 17
7.3.1. GCC and Build Tools 17
7.3.2. MQTT Dependencies (Mosquitto) 17
Project Structure 18

7.4 18
7.4.1. Gateway Project: 18
7.4.2. Device Project (One per Device): 19
7.5. Configuration 19
7.5.1. Gateway (poc_config.h) 20
7.5.2. Each Device (poc_config.h) 20
7.6. Target Environment 20
7.7. Building the PoC 21
7.8. Running the PoC 21
7.8.1. Option 1 – Normal Flow 22
7.8.2. Option 2 — Fault Injection 24
7.8.3. Option 3 — Fault Injection: Error Management Stress 26
7.9. Notes and Limitations 28

1. IKERLAN

IKERLAN is a leading knowledge transfer technological centre providing competitive value to companies. We seek for excellence in R&D&i, thanks to the continuous adaptation to the needs of our customers and the proximity with the business reality. Faithful to our mission, we have been working daily since 1974 to develop solutions that allow our customers to become more and more competitive. We are a cooperative member of the MONDRAGON Corporation and the Basque Research and Technology Alliance (BRTA).

Thanks to a unique cooperation model, which combines technology transfer activities, internal research and training of highly qualified personnel, IKERLAN is currently the trusted technological partner of major companies in the country. To meet our goal, we are structured in two technological specialisation units:

- ELECTRONICS, INFORMATION AND COMMUNICATION TECHNOLOGIES
- ENERGY AND MECHATRONICS

IKERLAN is a centre that is dynamic and open to the world. We are an accredited agent of the Basque Network of Science, Technology and Innovation and of the Spanish Federation of Technology Centres. We have a major cooperation network integrated by renowned European centres and universities, with which we conduct activities of research and training of researchers.

2. SDK OVERVIEW

2.1. Software Architecture overview

The SDK is designed following a **layered architecture** approach. This design separates responsibilities, protects the public API, and ensures full portability across platforms such as PC and embedded microcontrollers (MCUs).

Each layer has a **clear and well-defined role**, allowing the SDK to evolve internally without impacting user applications.

The SDK is divided into the following layers: Application, Middleware, Services, Utils and Portability.

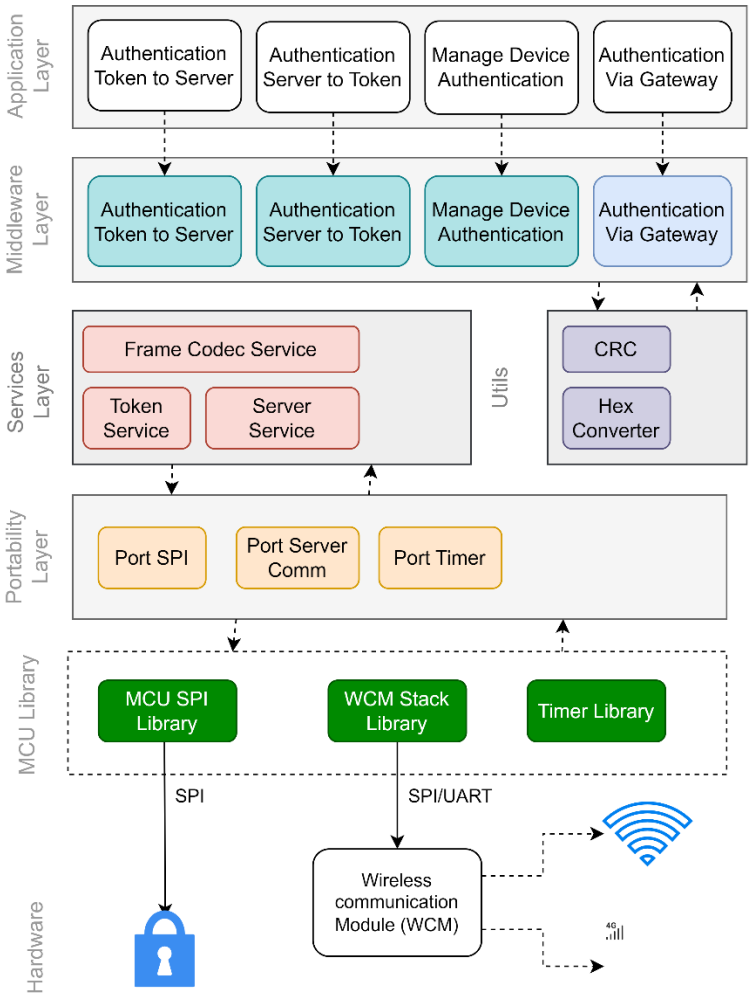


Figure 1. SDK Layered Architecture

2.2. Supported products

The SDK runs on the following set-up:

Set-up	Port_SPI	Port_COMM
virtualized environment	USB Sandgrain	PC MQTT
MCU 1		
MCU 2		

3. SDK LAYERS

3.1. Application Layer

The Application Layer is the highest-level layer of the SDK and represents the entry point for the client application. It exposes the public SDK API and is responsible for controlling the execution of the Middleware state machine until an operation reaches a final state.

The Middleware layer operates as a state machine — each call to a Middleware function advances the operation by one step and returns either a final status or SGR_MW_BUSY if the operation is still in progress. The Application Layer handles this by iterating the Middleware call in a loop until a conclusive result is obtained, then translating that result into the corresponding sgr_sdk_status_t code returned to the client.

This design gives the client full flexibility over how the function is consumed. As shipped, each API function runs as a blocking call that only returns when the operation completes. However, since the loop is entirely contained within this layer, a client with specific concurrency requirements can replace or wrap this behavior — for example, driving the iteration from a task scheduler or an event loop — without modifying any lower layer.

The API functions exposed by this layer differ depending on the compilation mode:

DEVICE_MODE:

- SGR_SDK_Initialize — initializes the SDK
- SGR_SDK_Authenticate_Via_GW — runs the full authentication flow through the Gateway

GATEWAY_MODE:

- SGR_SDK_Initialize — initializes the SDK
- SGR_SDK_Authentication_Token_To_Server — authenticates the token against the server
- SGR_SDK_Authentication_Server_To_Token — authenticates the server against the token
- SGR_SDK_GW_Handle_Device_Auth — handles concurrent authentication requests across all connected device channels

The full specification of each function — parameters, return values, and usage notes — is covered in the SDK Public API section.

3.2. Middleware Layer

The Middleware Layer implements the authentication protocol logic and acts as the execution engine of the SDK. It is fully platform-independent and contains no hardware-specific code.

This layer is implemented as an explicit state machine. Each call to a Middleware function advances the operation by one step and returns either a conclusive status (SGR_MW_OK, SGR_MW_AUTH_FAILED, SGR_MW_COMM_ERROR, SGR_MW_ERROR) or SGR_MW_BUSY if the operation is still in progress. The Application Layer is responsible for driving this iteration until a final result is obtained.

The Middleware Layer exposes different functions depending on the compilation mode.

DEVICE_MODE:

- **sgr_mw_init:** initializes the middleware and reads the Token ID from the crypto chip
- **sgr_mw_authenticate_via_gw:** drives the full Device↔Gateway authentication flow

GATEWAY_MODE:

- **sgr_mw_init:** initializes the middleware and reads the Token ID from the crypto chip
- **sgr_mw_authentication_token_to_server:** drives Phase 1 of the Gateway's own authentication against the Server (Token→Server)
- **sgr_mw_authentication_server_to_token:** drives Phase 2 of the Gateway's own authentication against the Server (Server→Token)
- **sgr_mw_gw_authenticate_device(int device_idx):** orchestrates the full DEVICE↔GW↔SERVER authentication flow for a specific device channel

Authentication Flow (GATEWAY_MODE)

When the Gateway handles a device authentication request via **sgr_mw_gw_authenticate_device**, the state machine progresses through the following sequence:

1. Receive Auth Request: The Gateway receives the authentication request from the Device, which includes its CRC identifier. The CRC is resolved against the auth_db to retrieve the corresponding Token ID. If the Device is not registered, the flow is rejected immediately.
2. Generate Challenge Word (CW1): The Gateway generates a random 32-byte Challenge Word and stores it in the authentication context.
3. Send CW to Device: CW1 is sent to the Device, which computes a Response Word (RW1) using its internal crypto chip and returns it along with a CW echo (CW2).
4. Verify CW: the Gateway compares CW1 against the received CW2 at bit level. If the difference exceeds CW_BITFLIP_THRESHOLD, authentication fails immediately. If the difference is within the threshold, the flow continues. If there are minor differences, candidate CWs are built for potential use in Error Management.
5. Get RW from Server: The Gateway sends CW1 and the Token ID to the Server, which computes its own Response Word (RW2) using the same secret key.
6. Compare RW: The Gateway compares RW1 (from the Device) against RW2 (from the Server) at bit level. If the difference is within RW_BITFLIP_THRESHOLD, authentication succeeds. If not, the Error Management mechanism is triggered.

7. Error Management: If the RW comparison fails but CW candidates are available, the Gateway systematically tests each candidate CW against the Server to find a matching RW. If a candidate produces a match within the threshold, authentication succeeds. If all candidates are exhausted, authentication fails.
8. Send Result: The result (AUT_OK or AUT_NOK) is sent back to the Device and recorded in the auth_db.
9. Retry: If the operation failed and the retry limit (GW_MAX_RETRIES) has not been reached, the state machine resets and a new attempt begins from the start.

The following diagram illustrates the complete authentication flow:

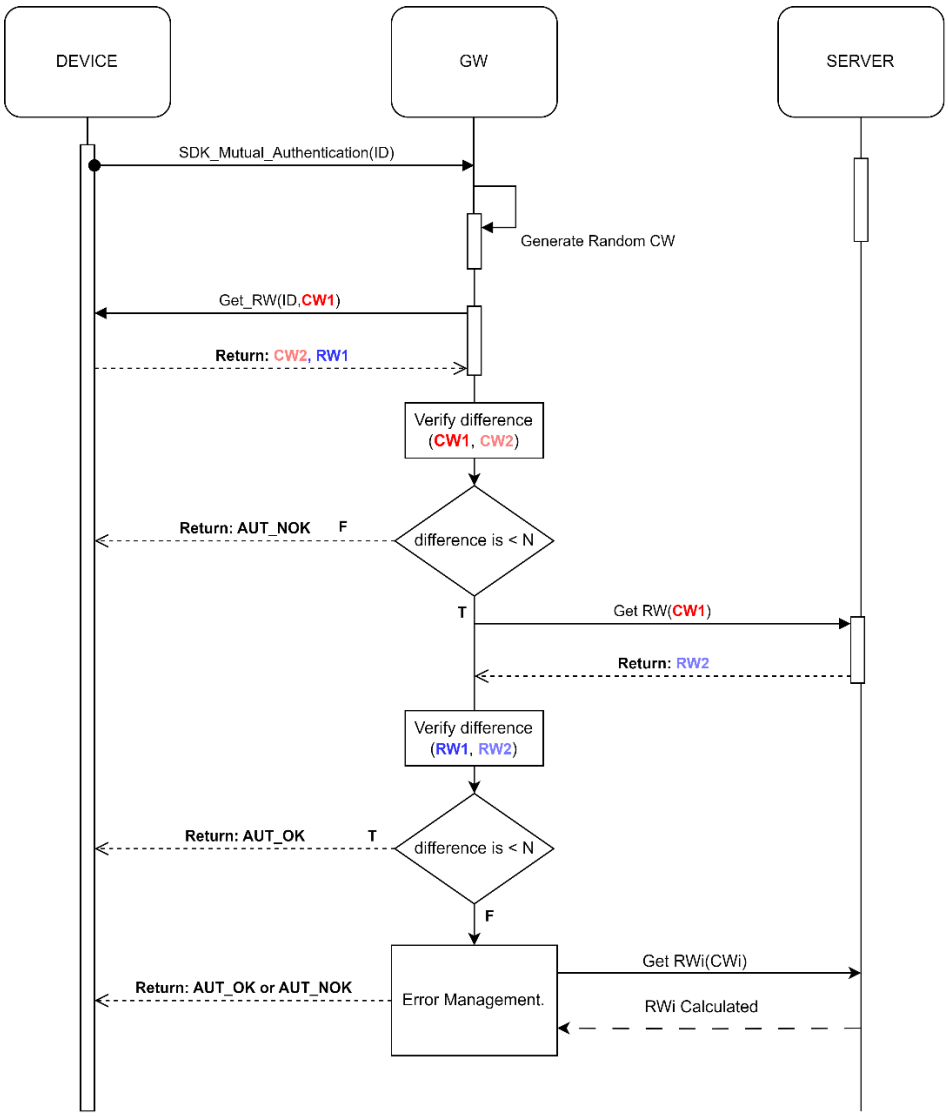


Figure 2. Sequence diagram for authentication with error handling in the GW.

3.3. Services Layer

The Services Layer provides the functional building blocks used by the Middleware Layer to perform concrete actions. Each service is focused on a specific responsibility and can be reused across different authentication flows.

The following services are available in both modes:

- **Frame Codec Service:** handles message framing, encoding, and decoding
- **Token Service:** Manages secure interaction with the crypto chip
- **Server Service:** Abstracts communication with the next node in the authentication chain. In GATEWAY_MODE this service communicates with the Server, and in DEVICE_MODE it communicates with the Gateway. The underlying transport implementation is provided by the Portability Layer and varies depending on the target platform.
- **Utils:** provides utility functions including CRC calculation and hexadecimal conversion.

The following services are available in GATEWAY_MODE only:

- **Device Service:** Handles communication between the Gateway and the connected Devices.
- **Database Service:** Manages the authentication database, including Device registration and CRC-to-TID resolution

3.4. Portability Layer

The **Portability Layer** abstracts platform-dependent capabilities required by the SDK.

This layer provides unified interfaces for:

- SPI communication
- Server communication interfaces
- Timing and delay mechanisms

By isolating platform-specific implementations, the Portability Layer allows the same SDK codebase to run on different platforms without modification.

Porting the SDK to a new platform requires changes only in this layer.

3.5. Client Platform Libraries

The SDK relies on **platform-specific libraries provided by the client** or by the target environment.

These libraries are **not part of the SDK architecture** and are not considered SDK layers.

Typical examples include:

- MCU SPI drivers
- Wireless communication or modem stacks

- Timer or OS abstraction libraries

These libraries are used **exclusively by the Portability Layer** to access platform capabilities. No other SDK layer depends on or interacts with client platform libraries directly.

4. SDK PACKAGE DESCRIPTION

This chapter describes the **package structure of the SGR SDK** and explains how the source code is organized according to the layered architecture.

The SDK package is divided into:

- docs
- examples
- include → Public SDK headers
- port → Port layer interface headers
- src → SDK source code

5. SDK PUBLIC API

This chapter describes the **public application programming interface (API)** exposed by the SGR SDK. The SDK API provides **high-level functions** that allow the client application to perform mutual authentication operations without requiring knowledge of internal protocols, cryptographic mechanisms, or hardware access. All SDK API functions are **synchronous** and maintain internal state within the SDK.

5.1. API Usage Model

The SDK follows a stateful usage model. The available functions and the typical usage flow differ depending on the compilation mode.

DEVICE_MODE:

- Initialize the SDK
- Perform authentication flows

GATEWAY_MODE:

- Initialize the SDK
- Authenticate the Gateway itself against the Server
- Handle incoming Device authentication requests

5.2. Status Codes

All SDK API functions return a value of type sgr_sdk_status_t. The client application must always check the return value to determine the result of the operation.

Value	Description
SGR_SDK_OK	Operation completed successfully
SGR_SDK_ERROR	Generic error
SGR_SDK_INVALID_PARAM	Invalid parameter provided
SGR_SDK_NOT_READY	SDK not ready or not initialized
SGR_SDK_TOKEN_ERROR	Error interacting with the crypto token
SGR_SDK_CRC_ERROR	Data integrity check failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_TIMEOUT	Operation timed out
SGR_SDK_INTERNAL_ERROR	Internal SDK error
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_NOT_REGISTERED	Device not registered in the Gateway
SGR_SDK_REGISTRATION_FAILED	Device registration attempt failed
SGR_SDK_NO_DATA	No data available from Device

5.3. Initialize Function

- **Prototype:** *sgr_sdk_status_t SGR_SDK_Initialize(void)*
- **Description**

  Initializes the SGR SDK internal components and prepares the system for authentication operations. This function must be called **once** before invoking any other SDK API.

- **Parameters:** None
- **Return values:** Refer to Section 5.2 for the complete list of status codes.
- **Usage notes**
  - Must be called before any authentication function.
  - In GATEWAY_MODE, this function additionally initializes the authentication database and registers all devices from the whitelist before initializing the middleware.
  - In DEVICE_MODE, only the middleware is initialized.

5.4. Device Mode Functions

5.4.1. SGR_SDK_Authenticate_Via_GW

- **Prototype:** sgr_sdk_status_t SGR_SDK_Authenticate_Via_GW(void);
- **Description:** Performs the full Device authentication flow via the Gateway. The function drives the Middleware state machine until the operation reaches a final state.
- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_ERROR	Internal SDK error

- **Usage notes:**
  - Requires the SDK to be initialized.
  - The function blocks until the authentication flow completes.

5.5. Gateway Mode Functions

5.5.1. Token-to-Server Authentication

- **Description**

  Performs **token-to-server authentication**. This function executes the complete authentication flow, including:

  - Requesting a Challenge Word (CW) from the server
  - Computing the Response Word (RW) using the secure token
  - Sending the Response Word back to the server
  - Processing the authentication result

  All steps are handled internally by the SDK.

- **Prototype:**
  sgr_sdk_status_t SGR_SDK_Authentication_Token_To_Server(void)

- **Parameters:** None.

- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed

SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - The function blocks until the authentication flow completes.
  - Retry mechanisms are handled internally by the SDK.

5.5.2. Server-to-Token Authentication

- **Description**

  Performs **server-to-token authentication**.

  This function executes the reverse authentication flow, including:

  - Generating a Challenge Word from the token
  - Requesting and validating the Response Word
  - Reporting the authentication result
- **Prototype**

  sgr_sdk_status_t SGR_SDK_Authentication_Server_To_Token(void)
- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - Intended for mutual authentication scenarios.
  - Hardware and protocol details are fully abstracted.

5.5.3. SGR_SDK_GW_Handle_Device_Auth

- **Description:**

Processes incoming authentication requests from all connected Devices. This function iterates over all available device channels (up to MAX_DEVICES) and advances each channel's authentication state machine, allowing multiple Devices to be handled concurrently.

This function is designed to be called continuously in a loop from the client application. On each call it processes the current state of all channels and returns, giving the client full control over the execution loop and the ability to integrate additional logic between iterations if needed.

- **Prototype:**

sgr_sdk_status_t SGR_SDK_GW_Handle_Device_Auth(void);

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	All channels processed normally
SGR_SDK_COMM_ERROR	Critical and unrecoverable communication error
SGR_SDK_ERROR	Critical and unrecoverable internal error

Usage notes:

- Requires the SDK to be initialized.
- Must be called after the Gateway's own authentication against the Server has completed successfully.
- This function is intended to be called in a loop from the client application.

6. CONFIGURATION

The SGR SDK provides a centralized configuration file, sgr_config.h, which controls the internal behavior of the SDK. All parameters take effect at compile time. This file covers SDK mode selection, debug output, retry limits, and CRC validation.

Platform-specific configuration such as network parameters, broker settings, or hardware port assignments is not part of the SDK and must be handled in the application or PoC configuration file (poc_config.h).

6.1. SDK Mode

The SDK_MODE parameter defines the compilation mode and determines which functions and port implementations are included in the build:

```
#define SDK_MODE GATEWAY_MODE /* or DEVICE_MODE */
```

Value	Description
DEVICE_MODE	Token node — authenticates via Gateway
GATEWAY_MODE	Gateway node — handles Device requests and Cloud communication

The SDK will emit a compile-time error if SDK_MODE is set to an invalid value.

6.2. Enabling Debug Output

The SDK exposes two independent debug flags, one per module:

MW_DEBUG controls the debug output of the middleware layer. ***PORT_COMM_DEBUG*** controls the debug output of the communication layer, including MQTT publish and receive operations.

To enable debug logging for a module, set its corresponding flag to ***1***. To disable it, either set it to ***0*** or comment out the define entirely:

```
#define MW_DEBUG 1 /* enable middleware debug logs */
#define PORT_COMM_DEBUG 1 /* enable communication debug logs */
```

When a flag is set to ***0*** or undefined, all log calls for that module are replaced by empty statements at compile time, producing no code and no runtime overhead. This makes it safe to ship production builds with debug flags disabled, as there is no performance penalty.

When both flags are enabled, the output will show the full communication flow for each step of the authentication process, including MQTT connection status, the JSON payloads transmitted and received, and the result of each operation. For example:

```
[PORT][MQTT] Connected
[PORT][MQTT] TX about to write | topic=sandgrain/v1/gw-001/req/requestCW | size=118 | data=[...]
[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (228 bytes) | data=[...]
[MW] ✓ CW received successfully
```

The ***[MW]*** prefix corresponds to **MW_DEBUG** and the ***[PORT][MQTT]*** prefix corresponds to ***PORT_COMM_DEBUG***, making it easy to identify which module generated each log line. Note that the MQTT transport layer is part of the Proof of Concept (PoC) implementation and is not a core component of the SDK itself. In a production integration, the communication layer would be replaced by the target platform's own transport mechanism, and the ***[PORT][MQTT]*** logs would reflect that implementation instead.

6.3. Retry Configuration

The ***MAX_RETRIES*** parameter defines how many times the SDK will retry a failed operation before reporting an error. A value of ***0*** means no retries will be performed and only one attempt will be made. If the client sets a negative value, the SDK will automatically override it and enforce the minimum of ***0***.

```
#if MAX_RETRIES < 0
        #warning "MAX_RETRIES is less than 0, forcing to 0"
        #undef MAX_RETRIES
        #define MAX_RETRIES 0
#endif
```

This validation happens at compile time and will emit a compiler warning to notify the developer that the value was corrected, so the behavior is never silent.

Important: The parameter validation block at the bottom of ***sgr_config.h*** must not be modified. It exists to guarantee safe minimum values are always enforced regardless of what the client configures above it.

6.4. CRC Validation

The SDK implements CRC16 validation for all received messages to ensure data integrity. This feature can be disabled for testing or debugging purposes.

Configuration

In ***sdk_sgr_config.h***:

```
#define CRC_ENABLE 1 // 1 = Enable, 0 = Disable
```

7. PROOF OF CONCEPT (POC)

This chapter describes the Proof of Concept provided with the SGR SDK. The PoC is a reference implementation used for validation, testing, and demonstration purposes. It is not a mandatory component of the SDK.

7.1. PoC Scope and Purpose

The PoC validates the following aspects of the SDK:

- Correct initialization of both the Device and Gateway executables
- Mutual authentication flow between Device, Gateway and Server
- Concurrent handling of multiple Device authentication requests by the Gateway
- Fault injection and Error Management mechanisms

7.2. Architecture

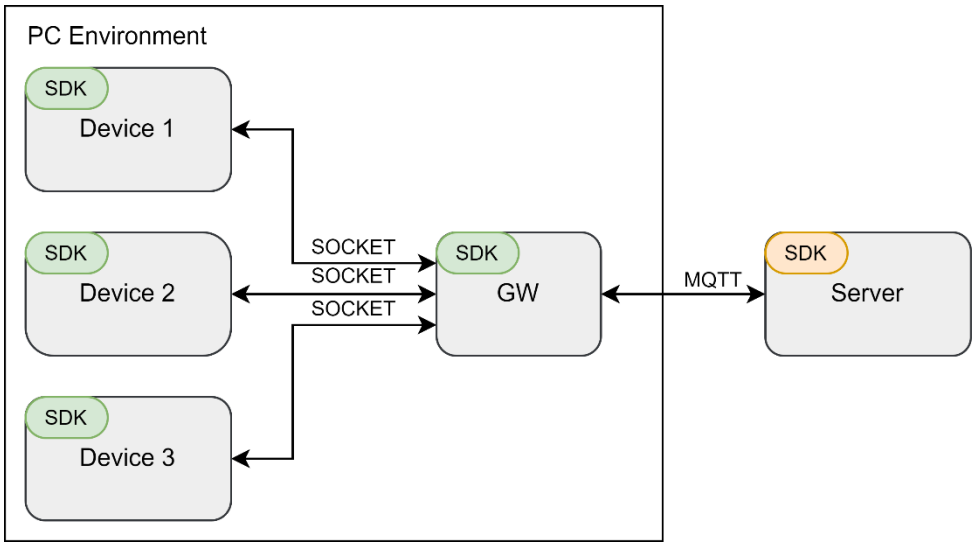


Figure 3. PoC Architecture

The PoC runs entirely in a PC environment and consists of four independent executables:

- **3 Device executables** — each compiled in DEVICE_MODE, each with its own crypto token connected via USB/SPI
- **1 Gateway executable** — compiled in GATEWAY_MODE, connected to all three Devices via TCP socket and to the Server via MQTT

Each Device connects to the Gateway on a dedicated TCP socket port. The Gateway listens on three consecutive ports starting from PORT_BASE:

Executable	Socket Port	Description
Device 1	8080	Connects to GW channel 0
Device 2	8081	Connects to GW channel 1
Device 3	8082	Connects to GW channel 2

7.3. PoC Dependencies

This section lists the tools and libraries required only for building and running the PoC. These dependencies are not required to use the SDK on other platforms (e.g., embedded MCUs), where equivalent platform-specific implementations are expected.

7.3.1. GCC and Build Tools

The PoC is written in C and requires a standard GCC toolchain.

Install the required build tools using:

sudo apt update

sudo apt install build-essential

This installs:

- GCC compiler
- Make build system
- Standard C libraries and headers

7.3.2. MQTT Dependencies (Mosquitto)

The PoC uses MQTT for server communication.

The Mosquitto client development library is required:

sudo apt install libmosquitto-dev

This library provides:

- MQTT client headers

MQTT client library used during linking

7.4. Project Structure

The PoC consist of two separate projects, one for the Gateway and one for the Device.

7.4.1. Gateway Project:

PoC_SGR/

- **.vscode/** → VSCode debug configuration
  - launch.json → Debug settings (Linux + Windows)
  - tasks.json → Build tasks
- **app/** → PoC Application
  - **config_poc.h** → Gateway configuration
  - **maing_gw.c** → Gateway main application
  - Makefile → Build system
  - **port/** → Platform-specific implementations
    - port_comm.c → MQTT communication layer
    - port_comm_device.c →Device socket communication
    - port_delay.c →Timing functions
    - port_spi_usb.c → USB/SPI token interface
- **sdk_fw/** → SGR SDK (read-only)
  - **docs** → Documentation about SDK
  - **examples** → Some examples of PORT layer implementation
  - **include/** → Public SDK headers
  - **port/** → Port layer interface headers
  - **src/** → SDK source code
- **setup-debug.sh** → Debug configuration script (Linux only)

7.4.2. Device Project (One per Device):

PoC_SGR/

- **.vscode/** → VSCode debug configuration
  - launch.json → Debug settings (Linux + Windows)
  - tasks.json → Build tasks
- **app/** → PoC Application
  - **config_poc.h** → Gateway configuration
  - **maing_device.c** → Gateway main application
  - Makefile → Build system
  - fault_injection
    - fi_device.c → Fault injection implementation
    - fi_device.h → Fault injection conriguration
  - **port/** → Platform-specific implementations
    - port_comm.c → MQTT communication layer
    - port_comm_device.c →Device socket communication
    - port_delay.c →Timing functions
    - port_spi_usb.c → USB/SPI token interface
- **sdk_fw/** → SGR SDK (read-only)
  - **docs** → Documentation about SDK
  - **examples** → Some examples of PORT layer implementation
  - **include/** → Public SDK headers
  - **port/** → Port layer interface headers
  - **src/** → SDK source code
- **setup-debug.sh** → Debug configuration script (Linux only)

Key Points:

- The **sdk_fw/** directory contains the SDK core and should not be modified
- Configuration is done exclusively through **sdk_fw/include/sdk_sgr_config.h**
- The app/ directory contains the PoC application and platform-specific code

7.5. Configuration

Before building the PoC, two configuration files must be set up for each executable:

sdk_sgr_config.h — defines the compilation mode. This is the only parameter that needs to be set here:

```
#define SDK_MODE DEVICE_MODE /* for Device executables */
#define SDK_MODE GATEWAY_MODE /* for Gateway executable */
```

poc_config.h — defines all environment-specific parameters. This is the only file that needs to be adapted to your hardware and network setup. Do not modify any other SDK file.

7.5.1. Gateway (poc_config.h)

```
#define MAX_DEVICES 3 /* number of device channels */
#define PORT_BASE 8080 /* base TCP port — GW listens on 8080, 8081, 8082 */
```

Additionally configure the MQTT broker parameters:

```
#define MQTT_BROKER "172.16.124.248" /* Broker IP or hostname */
#define MQTT_PORT 1883 /* Broker port */
#define MQTT_USER "admin" /* Broker username */
#define MQTT_PASS "changeme" /* Broker password */
#define MQTT_KEEPALIVE 60 /* Keepalive in seconds */
```

Configure the USB port of the crypto token for each Device:

Windows:

```
#define USB_COM_PORT 18 // Change to your COM port number
```

Linux:

```
#define USB_TTY_PORT 0   // Change to your ttyACM number
```

7.5.2. Each Device (poc_config.h)

```
#define PORT 8080 /* socket port matching the GW channel */
#define SERVER_IP "127.0.0.1"
```

Windows:

```
#define USB_COM_PORT 18 // Change to your COM port number
```

Linux:

```
#define USB_TTY_PORT 0   // Change to your ttyACM number
```

7.6. Target Environment

The PoC has been developed and tested in the following environment:

- Operating System: Windows (MinGW64 / MSYS2). Linux support is expected but has not been validated.
- Execution mode: 4 independent user-space processes (1 GW + 3 Devices)
- Communication: TCP Socket (Device ↔ GW) and MQTT over TCP/IP (GW ↔ Server)
- Hardware: SGR crypto token connected via USB/SPI (one per Device)
- Build system: Make with MinGW64 toolchain

Porting or executing the PoC on a different environment may require adaptations outside the scope of this document.

7.7. Building the PoC

- Step 1 — Build the Gateway:

  cd poc_sdk_gateway/app

  make

- Step 2 — Build each Device:

  cd poc_sdk_device_1/app

  make

  cd poc_sdk_device_2/app

  make

  cd poc_sdk_device_3/app

  make

If the build completes successfully the following executables will be generated in each app/ directory:

Executable	Description
Poc_sdk_gateway/app/test_gw.exe	Gateway executable
poc_sdk_device_1/app/test_device.exe	Device 1 executable
poc_sdk_device_2/app/test_device.exe	Device 2 executable
poc_sdk_device_3/app/test_device.exe	Device 3 executable

Clean build:

make clean && make

7.8. Running the PoC

The Gateway must be started first and must be fully initialized before launching any Device. Open a separate terminal for each executable.

Terminal 1 – Start the Gateway

```
cd poc_sdk_gateway/app
./test_gw.exe
```

The Gateway is ready then the following output appears:

```
✓GW SDK initialized OK
[ AUTH ] Waiting for devices...
```

Terminal 2, 3, 4 – Start each Device:

```
cd poc_sdk_device_X/app
./test_device.exe
```

Each device presents an interactive menu with three test options:

```
=====================================================
 Select test option:
 1. Normal flow (no fault injection)
 2. Fault injection (CW: 2 bits, CW echo: 1 bit, RW: 1 bit)
 3. Fault injection — Error Management stress (CW: 3 bits, CW echo: 2 bits)
=====================================================
 Option:
```

The three Devices can be launched independently and do not need to be started simultaneously. Each Device connects to its assigned GW channel based on the PORT value configured in its poc_config.h.

7.8.1. Option 1 – Normal Flow

This option executes the complete authentication flow without any fault injection. Both the CW and RW are transmitted without corruption, and the Gateway verifies them without needing to invoke the Error Management mechanism.

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2

[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [3/4] RW:
dc6c0db144ac810b9ead8aa438121236
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
dc6c0db144ac810b9ead8aa438121236
[MW] [CH1][DEBUG] CW1 original:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [CH1][DEBUG] CW2 received:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [CH1][DEBUG] Differing bit positions:
[MW] [CH1][4/6] CW verified — bit diff=0 ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
dc6c0db144ac810b9ead8aa438121236
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=0
[MW] [CH1][6/6] RW match — bit diff=0 — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

The Device sends its CRC identifier to the Gateway. The Gateway resolves the CRC against the authentication database, generates a random 32-byte Challenge Word (CW1) and sends it to the Device. The Device passes the CW to the crypto chip, which computes a Response Word (RW). The Device returns the CW echo and RW to the Gateway. The Gateway verifies that CW1 and the received CW echo are identical (bit diff = 0), then queries the Server for its own RW computation. Both RWs match exactly (bit diff = 0) and AUT_OK is returned to the Device.

7.8.2. Option 2 — Fault Injection

This option simulates communication channel errors by injecting controlled bit corruptions into the CW and RW exchanged between the Device and the Gateway. The purpose is to validate the Gateway's bitflip tolerance and Error Management mechanism under realistic fault conditions.

Target	Bits injected	Location
CW received by Device	2 bits	port_comm_receive
CW echo sent by Device	1 additional bit	port_comm_send
RW sent by Device	1 bit	port_comm_send

The corruptions are applied at the Port layer of the Device, simulating errors that could occur in a real communication channel.

Device Output:

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [3/4] RW:
9867a76cc1a236c5c1c5fb92edc48063
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][DEBUG] CW1 original:
324535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][DEBUG] CW2 received:
bac535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][DEBUG] Differing bit positions:
3 7 15
[MW] [CH1][4/6] CW verified — bit diff=3 — 7 candidates built ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
436188c7fa4cbc931c7bbcaba7bdbe8a
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=75
[MW] [CH1][6/6] RW mismatch — bit diff=75 > threshold=3
[MW] [CH1][ERROR MGT] Starting Error Management — 7 candidates
[MW] [CH1][ERROR MGT] Testing candidate 1 / 7
[MW] [CH1][ERROR MGT] Candidate 1 / 7
[MW] [CH1][ERROR MGT] CW candidate:
bac535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][ERROR MGT] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] RW_server:
e1ee646c6f2e84937c815a0b61f8bcf7
[MW] [CH1][ERROR MGT] bit_diff=56 — no match, trying next
[MW] [CH1][ERROR MGT] Candidate 1 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 2 / 7
[MW] [CH1][ERROR MGT] Candidate 2 / 7
[MW] [CH1][ERROR MGT] CW candidate:
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][ERROR MGT] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] RW_server:
9867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] bit_diff=1 — MATCH ✓
[MW] [CH1][ERROR MGT] Candidate 2 matched — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

The Device receives CW1 from the Gateway and the Port layer injects 2 bit corruptions, producing a corrupted CW. The crypto chip computes the RW using this corrupted CW. Before sending back to the Gateway, 1 additional bit is injected in the CW echo (total 3 bits different from CW1) and 1 bit is injected in the RW.

The Gateway detects 3 bits of difference between CW1 and the received CW echo (positions 3, 7 and 15) and builds 7 candidates. The initial RW comparison fails with a bit diff of 75, well above the threshold. Error Management is triggered and tests each candidate against the Server. Candidate 1 fails. Candidate 2 matches with bit diff = 1, within the threshold, and AUT_OK is returned.

7.8.3. Option 3 — Fault Injection: Error Management Stress

This option is designed to stress-test the Error Management mechanism by forcing the Gateway to exhaust a larger number of candidates before finding the correct match. The fault injection configuration is more aggressive than option 2, requiring the Gateway to test all 7 candidates before resolving the authentication.

Target	Bits injected	Location
CW received by Device	1 bit	Port_comm_receive
CW echo sent by Device	2 additional bits	Port_comm_send

Device Ouput:

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
7a6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
7a6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [3/4] RW:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][DEBUG] CW1 original:
fa6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [CH1][DEBUG] CW2 received:
7aea67b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [CH1][DEBUG] Differing bit positions:
7 15 23
[MW] [CH1][4/6] CW verified — bit diff=3 — 7 candidates built ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
0e216d55241cc859f2f12b42de7cce8b
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=69
[MW] [CH1][6/6] RW mismatch — bit diff=69 > threshold=3
[MW] [CH1][ERROR MGT] Starting Error Management — 7 candidates
[MW] [CH1][ERROR MGT] Testing candidate 1 / 7
[MW] [CH1][ERROR MGT] Candidate 1 / 7 — bit_diff=57 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 1 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 2 / 7
[MW] [CH1][ERROR MGT] Candidate 2 / 7 — bit_diff=63 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 2 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 3 / 7
[MW] [CH1][ERROR MGT] Candidate 3 / 7 — bit_diff=62 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 3 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 4 / 7
[MW] [CH1][ERROR MGT] Candidate 4 / 7 — bit_diff=75 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 4 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 5 / 7
[MW] [CH1][ERROR MGT] Candidate 5 / 7 — bit_diff=67 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 5 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 6 / 7
[MW] [CH1][ERROR MGT] Candidate 6 / 7 — bit_diff=60 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 6 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 7 / 7
[MW] [CH1][ERROR MGT] Candidate 7 / 7
[MW] [CH1][ERROR MGT] RW_device:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][ERROR MGT] RW_server:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][ERROR MGT] bit_diff=0 — MATCH ✓
[MW] [CH1][ERROR MGT] Candidate 7 matched — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

he Device receives CW1 from the Gateway and the Port layer injects 1 bit corruption. The crypto chip computes the RW using this corrupted CW. Before sending back to the Gateway, 2 additional bits are injected in the CW echo, resulting in a total of 3 bits different from CW1.

The Gateway detects 3 bits of difference (positions 7, 15 and 23) and builds 7 candidates. The initial RW comparison fails with a bit diff of 69. Error Management is triggered and systematically tests all 7 candidates against the Server. Candidates 1 through 6 all fail. Candidate 7 produces an exact match with bit diff = 0 and AUT_OK is returned, demonstrating that the Error Management mechanism correctly exhausts all available candidates before resolving the authentication.

7.9. Notes and Limitations

- All PoC-specific dependencies are limited to the Linux-based reference implementation.
- The PoC relies on the Portability Layer to access platform services.
- No changes to the Application, Middleware, or Services layers are required to run the PoC.
- The PoC should be considered a reference and validation tool, not a production application.